Python Training

An Industrial Training Report Submitted To

Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal

Towards Partial Fulfilment for the Award of

Bachelor of Technology
In
Computer Science and Engineering

Submitted By

Aastha Chaurasiya(0820CS231001)

Department of Computer Science and Engineering
Lakshmi Narain College of Technology, Bhawarasla, Sanwer Road, Indore

Training I/C
Mr.Sandeep
(CodSoft)

Submitted To:
Mr. Pradeep Pal
HOD-CSE/IT
LNCT Indore

CERTIFICATE

This is to certified that Training Report entitled ***"Python Training"*** *submitted by* ***Aastha Chaurasiya (0820CS231001)*** *is a satisfactory account of the bona fide work done in* ***'CodSoft'*** *and is recommended for the Industrial Training, carried out in 5th Semester towards the partial fulfillment for the award of the degree* ***Bachelor of Technology*** *in* ***Computer Science and Engineering*** *to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (MP.)*

Date: Endorsed
By:

Training I/C
Mr.Sandeep
(CodSoft)

Mr. Pradeep Pal
HOD-CSE/IT
LNCT Indore

TRAINING REPORT APPROVAL SHEET

The training work entitled ***"Python Training"*** *submitted by* ***Aastha Chaurasiya (0820CS231001)****, is approved for Industrial Training as partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering by* ***Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.).***

Internal Examiner **External Examiner**

STUDENT DECLARATION

I, ***Aastha Chaurasiya*** hereby declare that the presented report of Internship titled **"Python Training"** at **CodSoft** is uniquely prepared by me after the completion of three months of work/training at the **CodSoft**.

I also confirm that this report is prepared solely for my academic requirements and not for any other purpose. It will not be used in the interest of any third party or corporation.

Aastha Chaurasiya

(0820CS231001)

CSE (Department of Computer Science)
LNCT, Indore

C.ID: 6d53894

CodSoft

CERTIFICATE

OF COMPLETION

PROUDLY PRESENTED TO

Aastha Chaurasiya

has successfully completed 4 weeks of a virtual internship program in

Python Programming

with wonderful remarks at **CODSOFT** from 12/09/2025 to 12/10/2025

We were truly amazed by his/her showcased skills and invaluable contributions to the tasks and projects throughout the internship.

Founder

contact@codsoft.in www.codsoft.in Date: 12/10/2025

Acknowledgment

Every endeavor requires an indomitable urge, perseverance, and proper guidance, especially when it is most needed. Motivated to enhance my skills and knowledge, I embarked on this training program with hope and determination. The journey has been both enlightening and rewarding.

I express my heartfelt gratitude to my mentors, for their exceptional guidance, invaluable insights, and unwavering support throughout the training program. Their mentorship has been instrumental in shaping my understanding of Front-End Development and applying it to practical scenarios.

Additionally, I would like to thank **Mr. Pradeep Pal (HOD, CSE/IT)** and **Dr. Prakash Bharani (Principal, LNCT Indore)** for their support and for fostering an environment conducive to learning. My gratitude also extends to the teaching and non-teaching staff of the Department of Computer Science and Engineering/Information Technology for their assistance throughout this training period.

I am deeply indebted to **Shri. Suprabhat Chauksey (M.D, LNCT Indore)** for providing the facilities and infrastructure necessary to successfully complete this training.

Lastly, I express my profound gratitude to my parents and my entire family for their constant encouragement and support. Without their dedication, this training would not have been possible. I also thank everyone who contributed, directly or indirectly, to the successful completion of this training.

It has been a privilege to learn and grow through this training program, which has added immense value to my academic and professional journey.

Aastha Chaurasiya

(0820CS231001)

Executive Summary

This project details the end-to-end design, logic implementation, and deployment of the **History-Saving Calculator**, a Python-based utility engineered for high reliability and data persistence. Unlike standard volatile calculators, this system integrates a robust **File I/O backbone** that ensures every mathematical transaction is logged and retrievable. The application serves as a practical implementation of fundamental software engineering principles, including input sanitization, modular function design, and persistent state management. By utilizing a clean Command Line Interface (CLI), the project balances computational efficiency with an organized user experience tailored for academic and professional logging.

Table of Contents

1. **Training Report Approval Sheet**
2. **Certificate**
3. **Student Declaration**
4. **Certificate of Training (Internship)**
5. **Acknowledgement**
6. **Detailed Executive Summary**
7. **Comprehensive Introduction**
8. **Technical Background Study**
9. **Extended Requirement Analysis**
10. **Modular System Design**
11. **Technical Implementation Details**
12. **Rigorous Testing Protocol**
13. **Analysis of Results**
14. **Conclusion & Future Roadmap**

Introduction

The development of the **History-Saving Calculator** marks a transition from simple script-writing to creating a persistent software tool. In modern computing, a "calculator" is often expected to be more than a momentary arithmetic engine; it must act as a reference tool. This project leverages the **Python programming language** to create a lightweight yet powerful environment for basic calculus (Addition, Subtraction, Multiplication, and Division). The core innovation lies in the **asynchronous-style logging mechanism**, where the application separates the calculation logic from the storage logic, ensuring that data is preserved even if the user exits the session unexpectedly.

Background Study

The evolution of calculators has moved from mechanical devices to digital interfaces. However, a common limitation in basic CLI tools is the "Lost State" problem—where data disappears upon process termination. To solve this, this project explores **Data Persistence** through flat-file storage (`.txt` files). By studying how operating systems handle file streams, the application was designed to use the **Append ('a') mode**, which is a low-overhead way to manage databases without the complexity of SQL. This study also covers **Input Parsing**, where a string-based user input is decomposed into operands and operators, a core concept in compiler design and interpreter building.

Requirement Analysis

A multi-dimensional analysis was conducted to ensure the application meets both performance and usability standards:

Hardware Requirements

- **Processor:** Any modern CPU (1.0 GHz+) capable of running a Python interpreter.
- **Memory (RAM):** 2 GB minimum; the script is optimized for low memory footprints, typically consuming less than 50MB during active calculation.
- **Disk Space:** Negligible (under 1MB), though storage will grow slightly as the `history.txt` file accumulates data over years of use.

Software Requirements

- **Interpreter:** Python 3.8 or higher is recommended for optimal string handling and f-string support.
- **File System:** A standard NTFS, APFS, or ext4 file system that supports read/write permissions for the local directory.

Functional Requirements

- **Parsing Logic:** Must accept inputs in the specific format: `number operator number`.
- **Calculation Engine:** Must support floating-point arithmetic to ensure precision in division.
- **History Retrieval:** Users must be able to view their last *n* calculations in reverse chronological order.
- **Clean-up Utility:** A dedicated command to reset the `history.txt` file to zero bytes.

System Design

The system follows a **Functional Decomposition** model, where the program is broken into independent, reusable blocks:

1. **The Controller (`main`):** An infinite `while` loop that acts as the primary listener for user commands.
2. **The Parser (`calculate`):** A string processor that uses `.split()` to isolate variables and checks for syntax validity.
3. **The Persistence Layer (`save_to_history`):** A gateway function that opens a file stream, writes the formatted string, and closes the stream to prevent data corruption.
4. **The History Viewer (`show_history`):** A retrieval block that reads the file into a list of lines and applies a `reversed()` iterator for a "Latest-First" view.

Implementation

During the implementation phase, specific Pythonic patterns were used to enhance reliability:

- **Error Handling:** `try...except` blocks were used specifically around the `float()` conversion and the division operator to catch `ValueError` and `ZeroDivisionError`.
- **Formatting:** The use of f-strings (`f"{equation} = {result}\n"`) ensures that the history log is human-readable and consistently formatted.
- **Memory Efficiency:** Instead of keeping the entire history in the system RAM, the program only reads the file when the `history` command is explicitly called, keeping the footprint small.

Testing

The application underwent **Unit Testing** and **Integration Testing**:

- **Standard Arithmetic Test:** Inputting `10 * 5` yielded `50` and successfully wrote to the file.
- **Zero-Division Guard:** Inputting `10 / 0` triggered a custom error message: "Error: Division by zero is not allowed".
- **Format Integrity:** Inputting `10+10` (no spaces) triggered a validation error, prompting the user to use the correct `number operator number` format.
- **History Persistence:** After performing five calculations and restarting the computer, the `history` command successfully retrieved all five lines from the disk.

Result

The project resulted in a **production-ready CLI Calculator**. It successfully demonstrates that Python can be used to create tools that are both computationally accurate and data-aware. The user interface is minimalist but highly effective, providing immediate feedback for calculations and a reliable secondary storage system for long-term record keeping.

Conclusion

The **History-Saving Calculator** successfully bridges the gap between basic coding exercises and functional tool development. Through this project, I gained deep insights into **Python File I/O**, **String manipulation**, and **Defensive Programming**.

Future Roadmap

- **GUI Integration:** Developing a Tkinter or CustomTkinter interface to replace the CLI.
- **Scientific Support:** Adding advanced functions like square roots, trigonometry, and logarithms.
- **Timestamping:** Modifying the history log to include the exact date and time of each calculation.
- **Cloud Sync:** Implementing a feature to sync the `history.txt` file to a cloud service for multi-device access.

ing.